

DPS Project 2026

Distributed and Pervasive Systems Lab Course

Version 1.0

29th May 2026

1 Project Description

This project consists in the design and implementation of SMARTFAB, a simplified simulation of a smart manufacturing plant composed of multiple production lines equipped with telemetry systems. Figure 1 shows the overall architecture of the system.

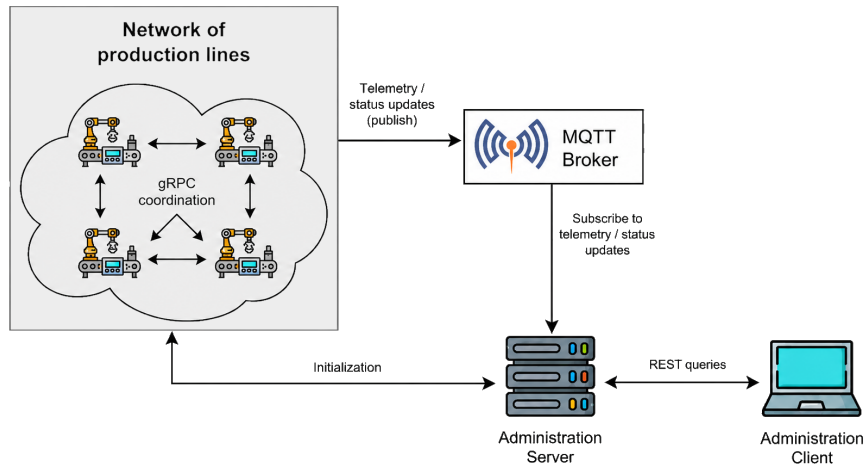


Figure 1: Overall architecture of SMARTFAB.

In modern manufacturing plants, production lines are increasingly monitored through distributed sensing systems. These systems collect data about the behavior of the production process, such as vibration levels, temperature, or defect indicators, and use this information to support quality-control and calibration operations.

In SMARTFAB, each production line is equipped with a telemetry system that periodically collects measurements and transmits them to an administration server. The administration server stores the received data and allows a plant operator, through an administration client, to monitor the state of the production lines.

When the data collected by a production line exceed critical values, the line needs to enter calibration mode. However, calibration operations temporarily stop the corresponding production line. To avoid stopping too many production lines at the same time, the factory allows only one line at a time to enter calibration mode. Therefore, production lines must coordinate before starting a calibration operation. If multiple lines require calibration concurrently, priority is given to the line whose telemetry data indicate the most critical

condition. Production lines also notify the administration server whenever they enter or exit calibration mode. This allows the administration client to show the current state of each line, including the telemetry data collected by the system, whether the line is currently in calibration, and whether it is waiting to enter calibration.

The goal of this project is to implement a peer-to-peer network of production lines, the administration server, and the administration client. The production lines must be able to collect telemetry data, transmit them to the administration server, detect critical conditions, notify calibration events, and autonomously coordinate with each other when multiple lines need to enter calibration mode.

2 General Operation

This section describes the general operation of the system. The later sections will provide a detailed explanation for each specific component.

2.1 Telemetry System

Each production line is equipped with a telemetry system that monitors the operating conditions of the line. The telemetry system periodically produces measurements related to the production process. These measurements are used to understand whether the line is operating normally or whether it may require calibration.

2.2 Production Line Status

Each production line can be in one of three states. A production line is `FullyOperational` when its telemetry data remain below the critical values defined by the system. When the collected data exceed such critical values, the production line requires calibration and enters the `WaitingForCalibration` state. Then, once the line obtains permission to start calibration, it enters the `UnderCalibration` state.

2.3 Calibration Access

Calibration temporarily stops the corresponding production line. To avoid stopping too many production lines at the same time, the factory allows only one line at a time to be in the `UnderCalibration` state. Therefore, when multiple production lines require calibration concurrently, they must coordinate to decide which line can enter calibration mode first. The priority is given to the line whose telemetry data indicate the most critical condition. Once the line enters the `UnderCalibration` state, the calibration operation is simulated for a random amount of time. All other lines trying to access calibration will remain in the `WaitingForCalibration` state. After the operation is completed, the line returns to the `FullyOperational` state.

2.4 Administration Monitoring

The administration server keeps track of the production lines currently active in the system. Production lines periodically transmit their telemetry data to the administration server. Moreover, whenever the state of a production line changes, the new state is communicated to the administration server. The administration client can query the administration server to monitor the state of the plant. In particular, it can be used to inspect the telemetry data collected from the production lines and to check the state of each line.

2.5 New Production Lines

The system allows the dynamic addition of new production lines at any time. When a new production line joins the system, it is registered by the administration server and becomes part of the network of production lines. From that moment on, it starts monitoring its telemetry data, reporting its state, and participating in the coordination required to access calibration mode.

3 Applications to Be Implemented

For this project, you are required to develop the following applications:

- **Production Line:** the process that represents a production line in the SMARTFAB plant. Each production line monitors its telemetry data, sends and receives messages from the other production lines when calibration is needed, and notifies the administration server about its measurements and state changes.
- **Administration Server:** a REST server that dynamically adds production lines to SMARTFAB, keeps track of the currently active production lines, receives telemetry data and state updates, and allows the administration client to compute statistics about the plant.
- **Administration Client:** a client that allows querying the administration server to obtain information about the currently active production lines, their telemetry data, and their current state.

Please note that each production line is a stand-alone process, and so, it must not be implemented as a thread. In the following, we provide more details about the applications that must be developed.

4 Production Line

Each production line runs a process that is responsible for:

- Acquiring telemetry data from its monitoring sensor;
- Detecting whether the collected data indicate a critical operating condition;
- Coordinating with the other production lines by using gRPC to access calibration mode;
- Sending telemetry data and operational-state updates to the administration server via MQTT.

4.1 Initialization

A production line is initialized by specifying:

- An integer ID;
- Listening address and port for communication with the other production lines;
- Administration server's address and port.

Once launched, the production line process must register itself with the system through the administration server. If its insertion is successful (*i.e.*, there are no other production lines with the same ID), the production line receives from the administration server the list of the other production lines already present in the network (*i.e.*, address and port number of each line).

Once the production line receives this information, it starts acquiring data from its monitoring sensor. If there are already other production lines in the network, the new production line presents itself to the other lines. Finally, the production line connects to the MQTT broker to publish telemetry data and operational-state updates.

4.2 Telemetry Data

Each production line collects telemetry data through a monitoring sensor. These data are used locally by the production line to detect critical operating conditions and are periodically sent to the administration server. The following sections describe how telemetry measurements are generated and how they must be processed before being used by the production line.

4.2.1 Monitoring Sensor

In a real manufacturing plant, a production line may be equipped with multiple sensors that monitor different aspects of the production process. For the sake of simplicity, in this project each production line is equipped with a single monitoring sensor that periodically detects its vibration level. The vibration level is used as an indicator of the operating conditions of the line. Every single measurement is characterized by:

- Vibration value;
- Timestamp of the measurement, expressed in milliseconds.

The generation of such measurements is produced by a `MonitoringSensor` simulator. In order to simplify the project implementation, it is possible to download the code of the sensor directly from the page of the course on *MyAriel*, under the “Project” section. The code of the sensor must be added as a package to the project, and it must not be modified.

During the initialization step, each production line launches the sensor thread that will generate the measurements for the monitoring sensor. Each `MonitoringSensor` simulator is a thread that consists of an infinite loop that periodically generates, with a pre-defined frequency, the simulated measurements. The `MonitoringSensor` simulator provides the following methods:

- `startMeasuring()`: starts the generation of measurements if the sensor has not started yet, or resumes it after a calibration operation;
- `pauseMeasuring()`: pauses the generation of measurements while the production line is under calibration;
- `stopMeasuring()`: stops the generation of measurements and the simulator thread.

4.2.2 Sliding Window Processing

Measurements produced by the sensor must be added to a proper data structure. We only provide the interface `Buffer` of this data structure that exposes three methods:

- `void addMeasurement(Measurement m);`
- `List<Measurement> readAllAndClear();`
- `void clear().`

Thus, it is necessary to create a class that implements this interface. The sensor thread uses `addMeasurement()` to fill the data structure. Instead, `readAllAndClear()` is used by the production line to obtain the measurements stored in the data structure. At the end of a read operation, `readAllAndClear()` makes room for new measurements in the buffer.

Instead, `clear()` is used by the production line to remove all measurements currently stored in the buffer.

Specifically, you must process sensor data through the sliding window technique that was introduced in the theory lessons. You must consider a sliding window of 8 measurements, with an overlap factor of 50%. For each window the average value is computed to determine the production line operational state.

4.3 Operational State Transitions

Each production line can be in one of the following operational states:

- **FullyOperational**: the production line is fully operative and its telemetry data do not indicate a critical condition;
- **WaitingForCalibration**: the production line detected a critical condition and is waiting to enter calibration mode;
- **UnderCalibration**: the production line is currently in calibration mode.

The initial state of each production line is **FullyOperational**.

The following sections describe the conditions that trigger transitions between these states.

4.3.1 Critical Condition – From FullyOperational to WaitingForCalibration

Let $\bar{v}$ be the average vibration value computed on the sliding window. Let $\theta = 80$ be the critical vibration threshold defined for the system.

A production line detects a critical condition whenever:

$$\bar{v} > \theta$$

When this condition is verified, the production line changes its state from **FullyOperational** to **WaitingForCalibration** and starts the procedure required to enter calibration mode. While in **WaitingForCalibration** or **UnderCalibration**, newly computed telemetry averages do not trigger additional calibration requests.

4.3.2 Access to Calibration – From WaitingForCalibration to UnderCalibration

Calibration mode is a limited resource of the factory. To avoid stopping too many production lines at the same time, only one production line at a time can enter calibration mode. Production lines coordinate in a distributed and decentralized way to access calibration mode, without using the administration server as a coordinator. All communications required by the distributed synchronization algorithm must be handled through gRPC.

When a production line is in the **WaitingForCalibration** state, it must request access to calibration mode. While waiting to enter calibration mode, the production line pauses its monitoring sensor by invoking the `pauseMeasuring()` method and clears the sliding window.

The *criticality* level of a production line is computed as:

$$criticality = \frac{\bar{v} - \theta}{\theta}$$

where $\bar{v}$ is the latest average vibration computed on the sliding window and θ is the critical vibration threshold. Higher values of *criticality* indicate more severe operating conditions. The *criticality* level of a production line is computed when it enters the

`WaitingForCalibration` state and it is used to determine the priority of the production line to enter calibration. This value remains unchanged until the production line exits the `UnderCalibration` state. If two or more production lines have the same *criticality* level, the line with the highest ID has priority.

You must implement the *Ricart and Agrawala* distributed mutual exclusion algorithm adapted to the priority rule described above. Once a production line obtains permission to enter calibration mode, it changes its state to `UnderCalibration`. When a production line exits calibration mode, it must notify the production lines whose requests have been delayed according to the mutual exclusion algorithm.

4.3.3 Calibration – From `UnderCalibration` to `FullyOperational`

The calibration operation is simulated through a `Thread.sleep()`. The duration of the calibration operation is randomly generated in the range $[3, 7]$ seconds. During calibration, the production line cannot request another calibration operation.

Once the calibration operation is completed, the production line must resume its monitoring sensor by invoking the `startMeasuring()` method. Then, the production line returns to the `FullyOperational` state.

4.4 Communication with the Administration Server

Each production line communicates telemetry data and operational-state updates through MQTT. In this way, the information produced by the production lines is published as a stream of events, rather than being sent to a single predefined consumer. This reflects a realistic manufacturing scenario, where the same data may be consumed by multiple components, such as dashboards, alerting systems, predictive-maintenance modules, or data logging services.

For the sake of this project, the only component that must be implemented as a consumer of these MQTT messages is the administration server. The administration server subscribes to the relevant topics, stores the received information, and uses it to answer the queries submitted by the administration client.

4.4.1 Telemetry Data

Every 10 seconds, each production line has to publish the list of the vibration averages computed after the last publication.

This list of averages must be published together with:

- The ID of the production line;
- The timestamp in which the list was computed;
- The current operational state of the production line.

This communication takes place through a proper MQTT topic to which the administration server is subscribed.

4.4.2 Operational State Updates

Whenever the operational state of a production line changes, the production line must publish an operational-state update.

In particular, a state update must be sent when:

- The production line changes from `FullyOperational` to `WaitingForCalibration`;
- The production line changes from `WaitingForCalibration` to `UnderCalibration`;

- The production line changes from `UnderCalibration` to `FullyOperational`.

Each state update must include:

- The ID of the production line;
- The new operational state;
- The timestamp of the state change;
- The criticality level that caused the calibration request, if applicable.

This communication takes place through a proper MQTT topic to which the administration server is subscribed.

4.5 Leaving the Network

For the sake of simplicity, it is assumed that no production line, once entered the network, leaves it.

5 Administration Server

The administration server collects the IDs of the production lines registered to the system and receives from them telemetry data and operational-state updates. This information will then be queried by the administration client. Thus, this server has to provide different interfaces for:

- Managing the initialization of the production lines;
- Receiving telemetry data and operational-state updates from the production lines;
- Enabling the administration client to execute queries on the state of the plant.

5.1 Production Line REST Interface

When a production line starts, it must contact the administration server through this **REST** interface to request its insertion into the network. The server has to store the following information for each production line joining the network:

- Its ID;
- IP address;
- The port number on which it is available to handle the communication with the other production line processes.

A production line can be added to the network only if there are no other production lines with the same identifier. If the insertion succeeds, the administration server returns to the production line the list of production lines that have already joined the network. For each production line, the returned list must include the ID, the address, and the port number for communication.

5.2 Production Line MQTT Interface

The administration server must be able to receive telemetry data and operational-state updates from the production lines. This means that it has to subscribe to the proper MQTT topics used by the production lines.

Telemetry data are the averages computed by the production lines from the measurements produced by their monitoring sensors. The administration server must store these data so that they can be used to answer the queries submitted by the administration client.

Operational-state updates are published by production lines whenever their state changes. The administration server must use these updates to keep track of whether each production line is currently `FullyOperational`, `WaitingForCalibration`, or `UnderCalibration`.

During the development of the project, make sure that you correctly synchronize read and write operations performed on the telemetry data and on the operational states of the production lines. Indeed, the administration server can receive new telemetry data or state updates while the administration client is requesting it to perform some computations on such values.

5.3 Administration Client REST Interface

When requested by the administration client through the interface described in Section 6, the administration server must be able to compute and return the following information:

- The list of production lines currently in the network;
- The current operational state of each production line;
- The average vibration level of a specific production line between timestamp t_1 and timestamp t_2 .

The administration server must compute these values using the telemetry data and operational-state updates received from the production lines.

6 Administration Client

The administration client consists of a simple command-line interface that enables interaction with the REST interface provided by the administration server. Hence, this application prints a straightforward menu to select one of the services offered by the administration server described in Section 5 and to enter possible required parameters.

7 Simplifications and Restrictions

It is important to recall that the scope of this project is to prove the ability to design and build distributed and pervasive applications. Therefore, all the aspects that are not strictly related to communication protocols, concurrency, distributed synchronization, and sensory data management are secondary. Moreover, it is possible to assume that no nodes behave maliciously. On the contrary, you should handle possible errors in data entered by the user. Furthermore, the code must be robust: all possible exceptions must be handled correctly.

The project can be developed either in Java or in Kotlin. No other programming languages are allowed. Although the Java and Kotlin libraries provide multiple classes for handling concurrency situations, for educational purposes, it is mandatory to use only the methods and classes explained during the laboratory course. Therefore, any necessary synchronization data structures, such as locks, semaphores, or shared buffers, should be implemented from scratch and will be discussed during the project presentation.

Considering the communication between the nodes' processes, it is necessary to use the gRPC framework. If broadcast communications are required among nodes, these must be carried out in parallel and not sequentially.

During the project presentation, students will be asked to run all the production line processes locally, on the same machine. However, the code must not rely on this assumption. The project must be structured so that production lines can communicate over a real network by using actual IP addresses and ports.

8 MQTT Broker

The MQTT Broker on which SMARTFAB relies is Mosquitto, and is assumed to be online at the following address:

`tcp://localhost:1883`

Production lines use this Broker to periodically publish the telemetry data computed from the measurements collected by their monitoring sensor. The Broker is also used by the production lines to publish updates about their operational state. For example, production lines may use different topics to distinguish telemetry data from operational-state updates, possibly also organizing them by production line. The administration server subscribes to the MQTT topics used by the production lines in order to collect telemetry measurements and keep track of the current state of the plant.

9 Project Presentation

During the evaluation of the project, we will ask you to discuss some parts of the source code and we will also check if it runs correctly considering some tests. Moreover, we will ask you one or more theoretical questions about the theoretical content of the lab lessons. You will run your code on your machine, while the source code will be discussed on the examiner's machine.

You must submit the source code before discussing the project. For the submission, you should archive your code in a `.zip` file, renamed with your university code, *i.e.*, the *matricola*. For instance, if your university code is 012345, the file should be named `012345.zip`. Then, the zip file should be uploaded at:

`http://upload.di.unimi.it`

It will be possible to submit your project a week before the exam date. You must complete the submission strictly two days before the exam date at 11:59 PM. For instance, if the exam is on the morning of the 15th, you must complete the delivery before 11:59 PM of the 13th.

We suggest students to use, during their project discussion, the `Thread.sleep()` instructions to show how the synchronization problems have been correctly handled. We recommend analyzing all the synchronization issues and creating schemes to illustrate how the components of your project communicate. These schemes may represent the format of the messages and the sequence of the communications that occur among the components during the different operations involved in the project. Such schemes can be very helpful during the project discussion. It is not necessary to show the structure of the classes of your project, but only the main aspects regarding the synchronization and the communication protocols you have implemented.

During the presentation of the project, we will ask you to test your code by launching at least 4 or 5 nodes, the administration server, and a single administration client.

10 Optional Parts

In order to encourage the presentation of the project in the first exam sessions, the development of the first optional part becomes mandatory from the September session, included, while both the first and the second optional parts are mandatory from the January session, included.

10.1 First Optional Part: Controlled Exit of Production Lines

For this optional part, you must also consider the possibility of a production line leaving SMARTFAB. You can assume that each production line process terminates only in a controlled way, meaning that the production line explicitly exits the system.

A production line can leave the system at any time by entering the `exit` command into the command line of the process. If the production line is currently under calibration, it must complete the calibration operation before leaving the system. If the production line is waiting to enter calibration mode, it must cancel its pending calibration request before leaving the system.

When leaving the system, the production line process must follow these steps:

- Complete the current calibration operation, if any;
- Cancel its pending calibration request, if any;
- Notify the other production lines;
- Request the administration server to be removed from the system.

When a production line receives the notification that another production line has left the system, it must update its local view of the network. If the leaving production line had a pending calibration request, such request must be removed from the data structures used by the distributed mutual exclusion algorithm.

10.2 Second Optional Part: Uncontrolled Exit of Production Lines

For the second optional part, you can no longer assume that each production line process terminates only in a controlled way. Hence, all the distributed synchronization mechanisms implemented for the project must take into account that a production line could leave the system in an uncontrolled way, *i.e.*, without the ability to notify the administration server or the other production lines.

In particular, the system must handle the case in which a production line crashes while it is fully operational, while it is waiting to enter calibration mode, or while it is under calibration. The remaining production lines must detect that the crashed line is no longer reachable and update their local view of the network accordingly.

If a production line crashes while it is in the `WaitingForCalibration` state, the other production lines must discard its calibration request. If a production line crashes while it is in the `UnderCalibration` state, the remaining production lines must eventually allow another waiting production line to enter calibration mode. The administration server must also be informed that the crashed production line is no longer active in SMARTFAB.

11 Plagiarism

Students must develop the project independently. The reuse of code written by other students, or the submission of code that is not the result of individual work, is not allowed. The code developed for the project must not be published (*e.g.*, on GitHub or GitLab) at least until March of the next year. Cases of plagiarism will be handled according to the rules of the course and of the University.

12 Updates

If needed, the current text of the project will be updated. Changes in the text will be highlighted. Please regularly check *MyAriel* for new versions of the project.